

NATIONAL ENGINEERING CENTER
University of the Philippines

8.0

LLMOps

Eugene Rex L. Jalao PhD

Professor

Department of Industrial Engineering and Operations Research

University of the Philippines Diliman

eljalao@up.edu.ph

UP NEC

Module Outline

- Introduction to Agentic AI
  - Capstone Project Case Proposal Generation
- Deep Learning, Transformers and LLMs
  - Capstone Project Business Case Workflow
- Agentic Workflows
  - Case Study and Capstone Project Work
- Agentic RAGs
  - Case Study and Capstone Project Work
- Orchestration
  - Case Study and Capstone Project Work
- Stateful Multi-Agent Systems
  - Case Study and Capstone Project Work
- Evaluation
  - Case Study and Capstone Project Work
- **LLMOps**
  - **Capstone Project Presentation**

Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- Dash HTML and Core Components
- Dash Interactive Components
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

LLMOps

- **Definition:** LLMOps (Large Language Model Operations) refers to the practices, tools, and processes used to develop, deploy, evaluate, monitor, and maintain LLM-powered applications throughout their operational lifecycle.
- LLMOps extends software and machine learning operations to applications powered by large language models.
- It manages not only the model but also prompts, knowledge sources, tools, configurations, and application workflows.
- It ensures that LLM applications can be reliably deployed, evaluated, monitored, and improved.
- It introduces operational controls for performance, reliability, security, and quality.

https://mlflow.org/llmops

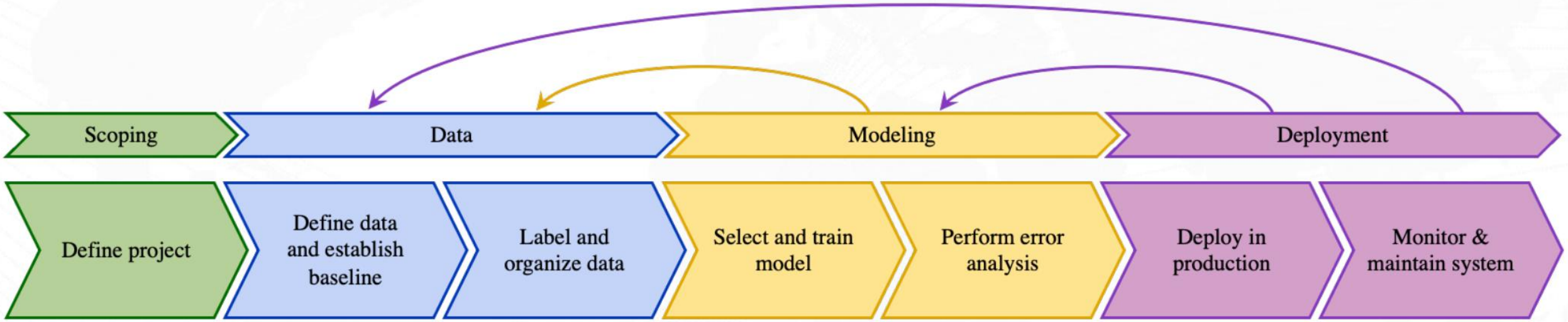
DevOps vs. MLOps vs. LLMOps

- **DevOps:** Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
- **MLOps:** Plan (i.e., business understanding) → Create (i.e., AI system architecture) → Condition (i.e., data conditioning) → Model (i.e., ML Modeling) → TestV&V →Package → Release → Configure → Deploy (i.e., CI/CD*) → Monitor
- **LLMOps:** Plan (i.e., business understanding) → Create (i.e., AI system architecture) → Condition (i.e., data conditioning) → Model (i.e., LLM Models) → TestV&V →Package → Release → Configure → Deploy (i.e., CI/CD*) → Monitor

DevOps vs. MLOps vs. LLMOps

- DevOps emphasizes reliable application delivery and infrastructure management.
- MLOps adds data pipelines, model training, validation, and performance monitoring.
- LLMOps adds prompt management, response evaluation, grounding, tracing, and controlled agent execution.
- LLMOps builds on DevOps and MLOps rather than replacing them.

LLMOps


Scoping
Data
Modeling
Deployment
Define project
Define data and establish baseline
Label and organize data
Select and train model
Perform error analysis
Deploy in production
Monitor & maintain system

Benefit of MLOps Process

- MLOps aims to unify the release cycle for machine learning and software application release
- It enables automated testing of machine learning artifacts (e.g., data validation, ML model testing, and ML model integration testing)
- MLOps enables the application of agile principles to machine learning projects
- Supports the development of machine learning models and datasets to build these models as first-class citizens within systems
- MLOps reduces technical debt across machine learning models
- MLOps must be a language-, framework-, platform-, and infrastructure-agnostic practice

LLMOps

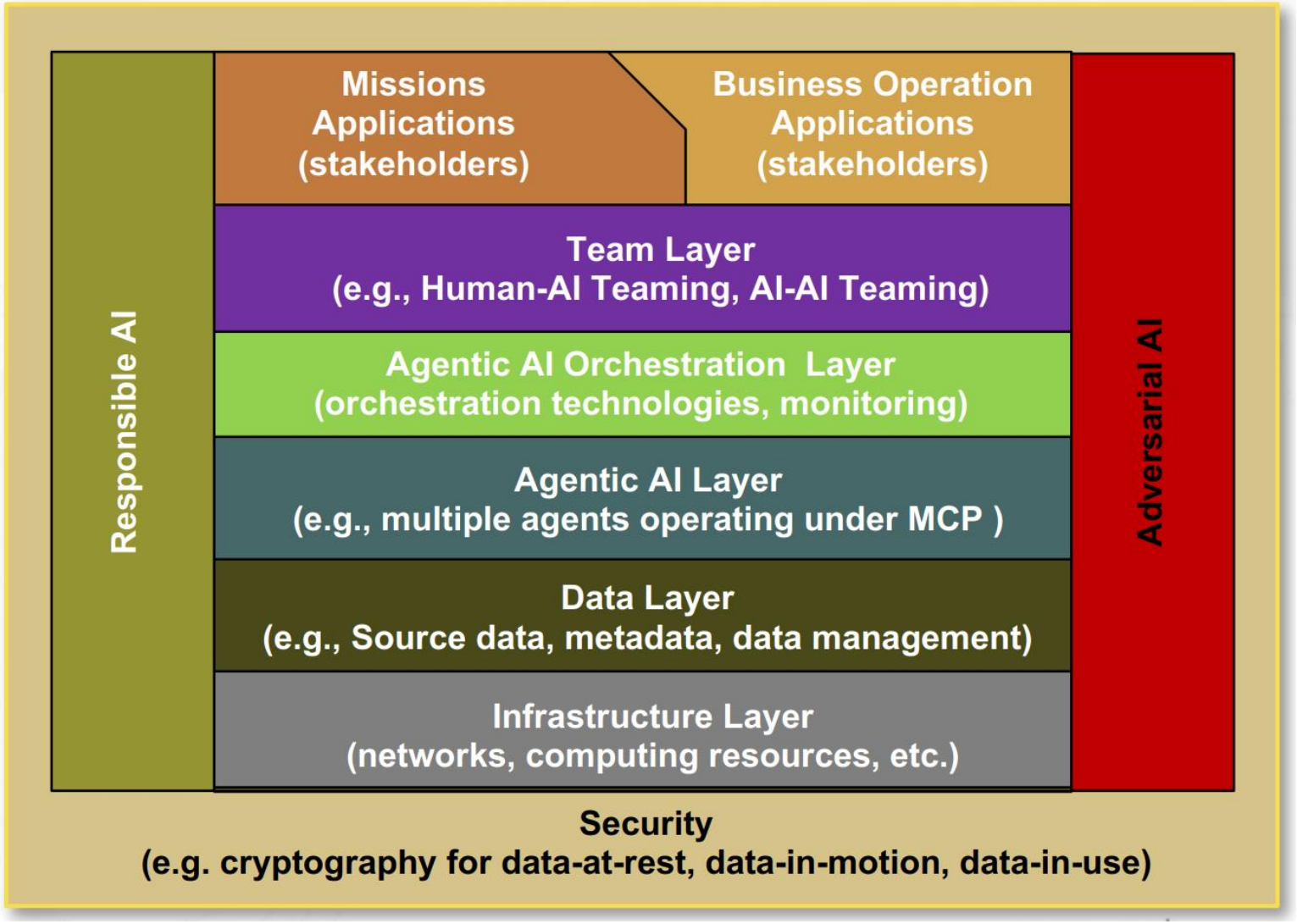
- A working Python prototype does not necessarily guarantee a reliable deployed application.

Prototype environment	Production challenges
A single user runs the agent in Spyder.	Multiple users may access the agent simultaneously.
Prompts and models are manually selected.	Versions and configurations must be controlled.
State is stored in temporary memory.	State must persist across requests and interruptions.
Tool calls are tested individually.	Tool execution needs permissions, validation, and logging.
Human approval is requested manually.	Approvals must be tracked, secured, and resumable.
Output is visually inspected.	Quality, failures, and performance must be continuously evaluated.

Outline for this Session

- Introduction to LLMOps
- **Architecture and Version Management**
- Deployment
- Dash HTML and Core Components
- Dash Interactive Components
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

Anatomy of an AI System

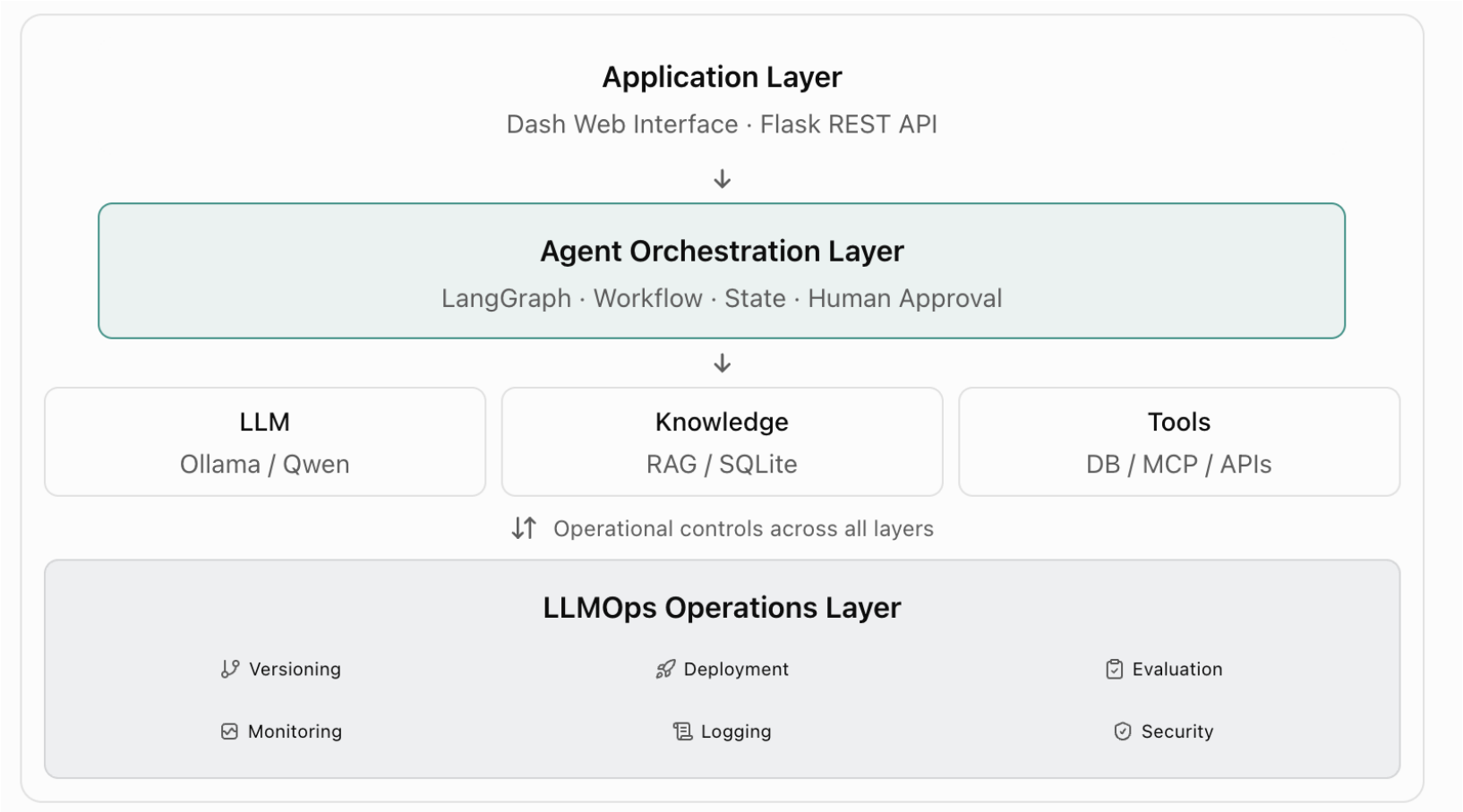


© David Martinez; MIT All Rights Reserved

LLMOps Architecture

- LLMOps manages the complete AI application, not only the underlying language model.
- The application layer provides interfaces, APIs, and access to agent capabilities.
- The agent orchestration layer coordinates models, tools, RAG, state, and human approval.
- The operations layer manages versioning, deployment, evaluation, logging, monitoring, and security.
- All layers must operate together to support reliable AI services.

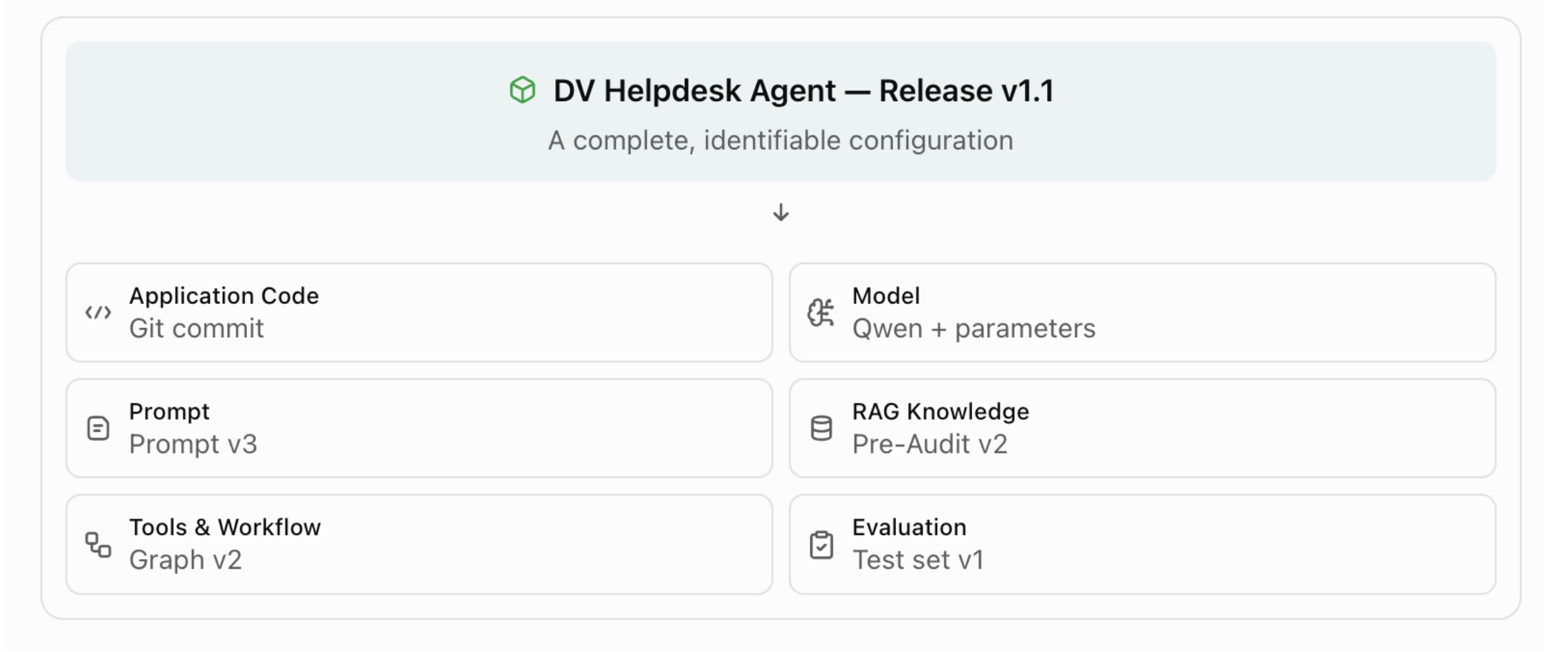
LLMOps Architecture



Version-Control in LLMOps

- **Application Code and Workflows:** Track changes to Python scripts, tool functions, LangGraph nodes, and routing logic.
- **LLM Models and Configurations:** Record the model identifier, exact model version, and generation parameters.
- **Prompts and Output Schemas:** Maintain versions of system instructions, prompt templates, and expected output structures.
- **RAG Knowledge Sources:** Version the source documents, embedding models, chunking settings, and retrieval indexes.
- **Tool Permissions and Guardrails:** Record approved tools, access policies, and human approval conditions.
- **Evaluation Datasets:** Maintain consistent test cases to compare performance across releases.

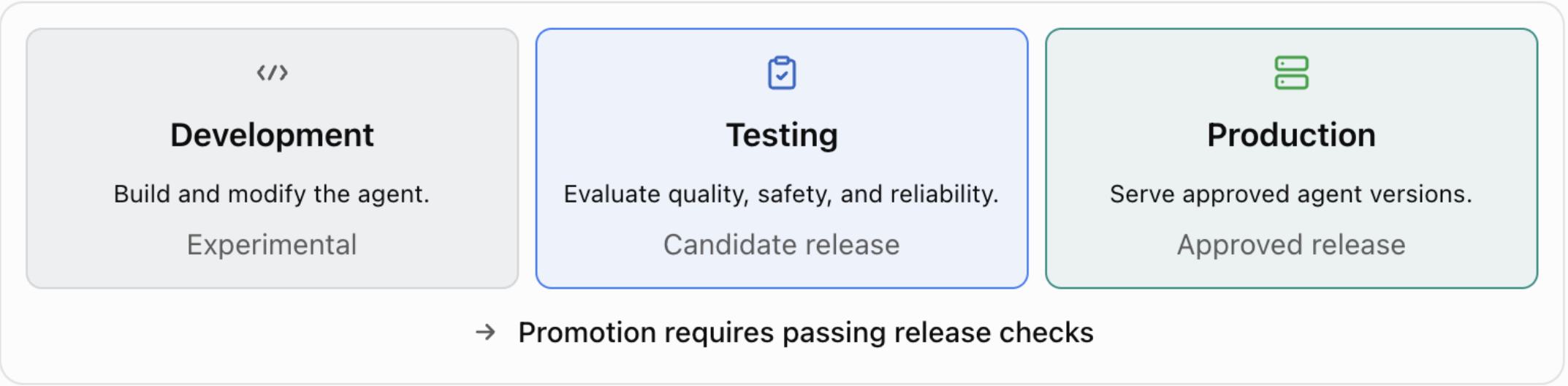
Version-Control in LLMOps



Development, Testing, and Production Environments

- **Development**: Developers experiment with prompts, tools, models, and workflow configurations.
- **Testing/QA**: Candidate releases undergo standardized evaluations, security tests, and regression checks.
- **Production**: Only approved versions are deployed for actual users and organizational workflows.
- **Reproducibility:** Each release should record the code, model, prompts, knowledge sources, dependencies, and configurations used.
- **Environment Separation:** Testing and experimentation must not accidentally modify production data or agent behavior.

Development, Testing, and Production Environments



Updates, Regression Testing, and Rollback

- **Changes** to models, prompts, RAG documents, and workflows can alter an agent's responses and actions.
- Every proposed update should create a **new version** rather than silently modifying the deployed version.
- Regression tests compare updated behavior against previously established requirements and expectations.
- Release decisions should consider response quality, tool accuracy, security, and compliance with business rules.
- Failed updates should be rejected, while problematic deployed releases should be rolled back to a previously approved configuration.

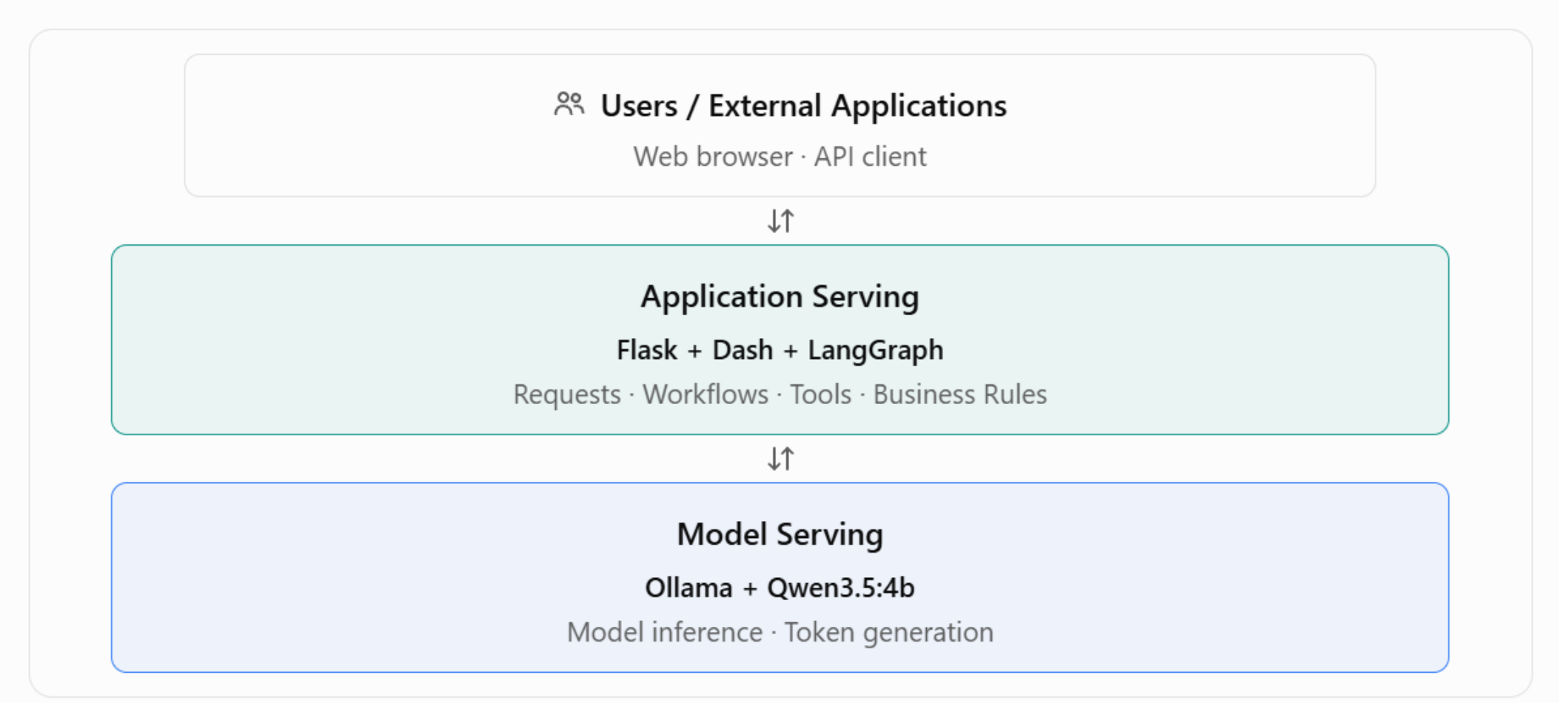
Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- **Deployment**
- Dash HTML and Core Components
- Dash Interactive Components
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

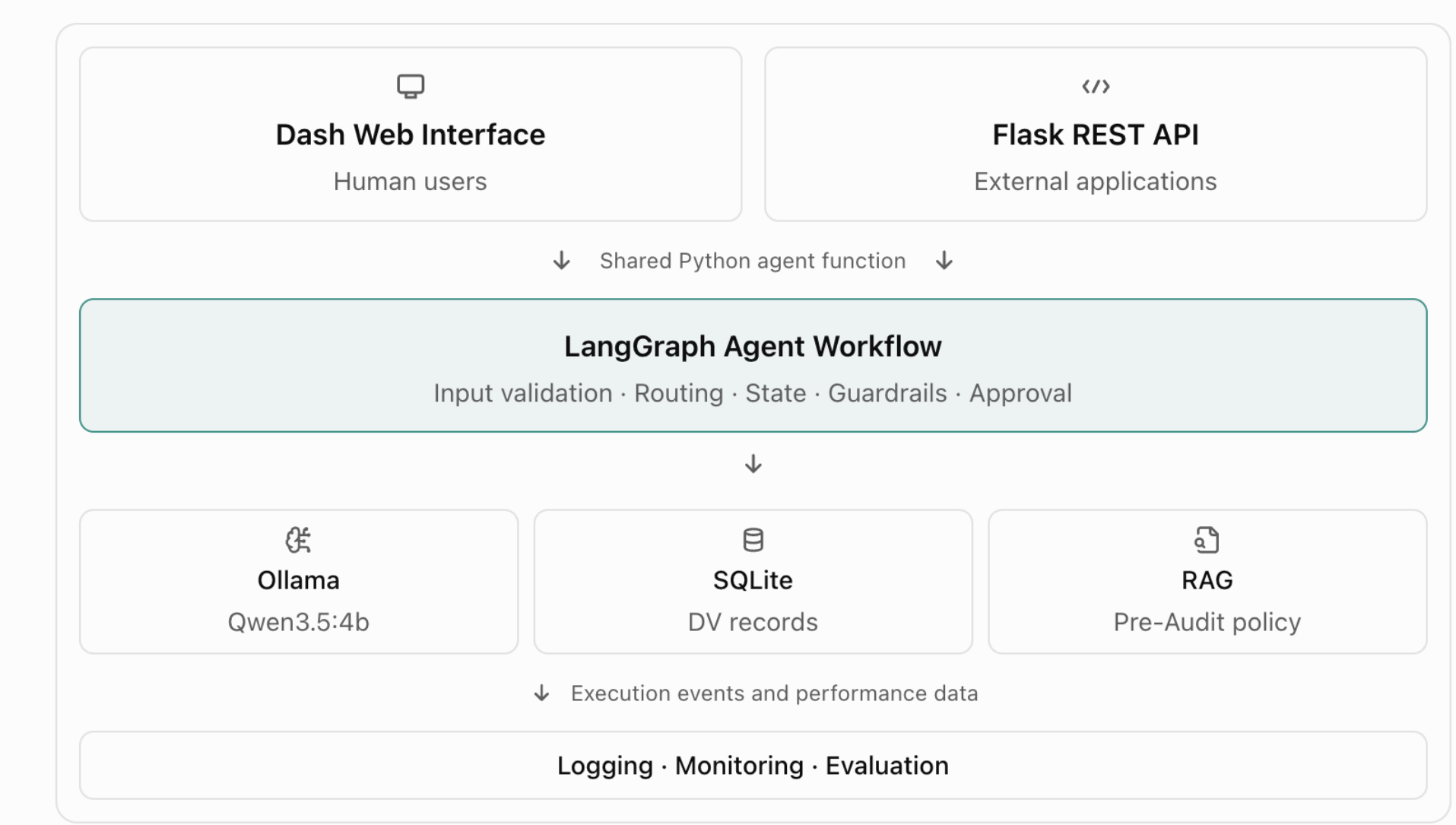
Model Serving vs. Application Serving

- Model serving makes an LLM available for inference through an API, accepting prompts and returning model-generated responses.
- Application serving exposes the complete AI application, including user interfaces, business logic, tools, RAG, and workflows.
- Example:
  - Ollama provides model serving, while Flask and Dash provide interfaces to the deployed Agentic AI application.
  - Model serving and application serving can operate on the same machine or on separate servers.

Model Serving vs. Application Serving



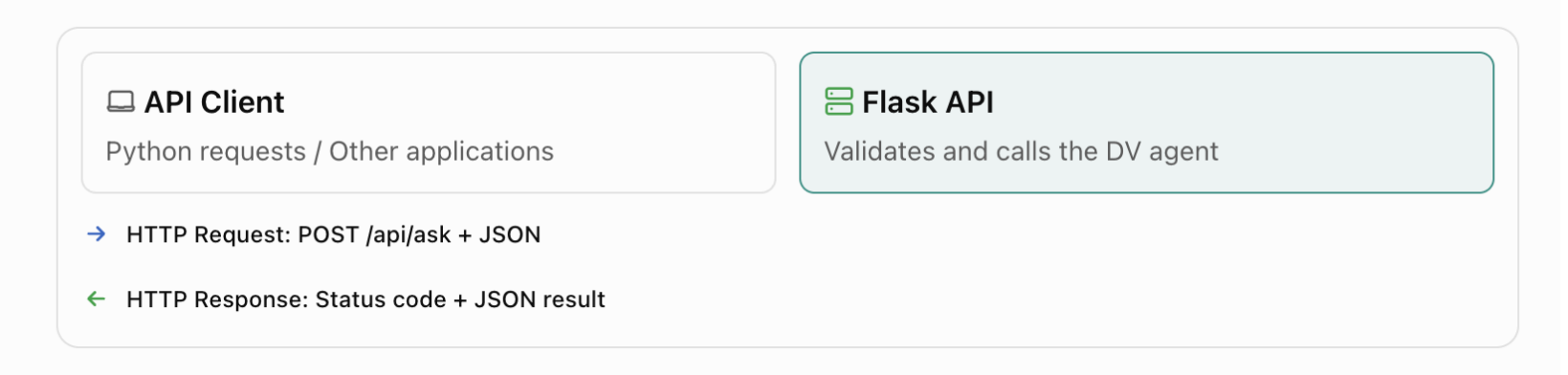
Architecture of a Deployed Agentic AI App



REST APIs and HTTP Requests

- A REST API allows software applications to interact through standardized HTTP requests and responses.
- API endpoints identify available services, such as submitting an inquiry or retrieving a workflow status.
- HTTP methods such as GET and POST specify the type of operation requested.
- JSON is commonly used to exchange structured data between clients and AI applications.
- Flask maps API endpoints to Python functions that process requests and return responses.

REST APIs and HTTP Requests

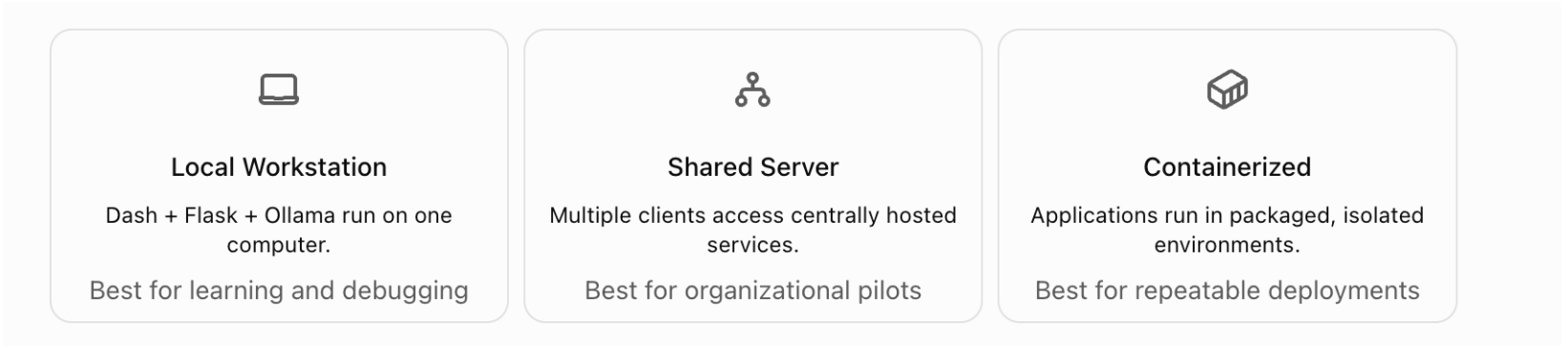


Method	Endpoint	Purpose
GET	/health	Check whether the application service is responsive.
POST	/api/ask	Submit a DV inquiry to the agent.
GET	/api/workflows/{id}	Retrieve the saved state of a workflow.
POST	/api/workflows/{id}/approve	Submit an authenticated approval decision.

Deployment Options

- Deployment decisions depend on compute capacity, number of users, security requirements, maintainability, and cost.
  - **Local deployment** runs the application and models on an individual workstation for development and testing.
  - **Shared server deployment** hosts the application or models centrally so multiple users can connect through a network.
  - **Containerized deployment** packages the application and its dependencies for consistent execution across environments.

Deployment Options

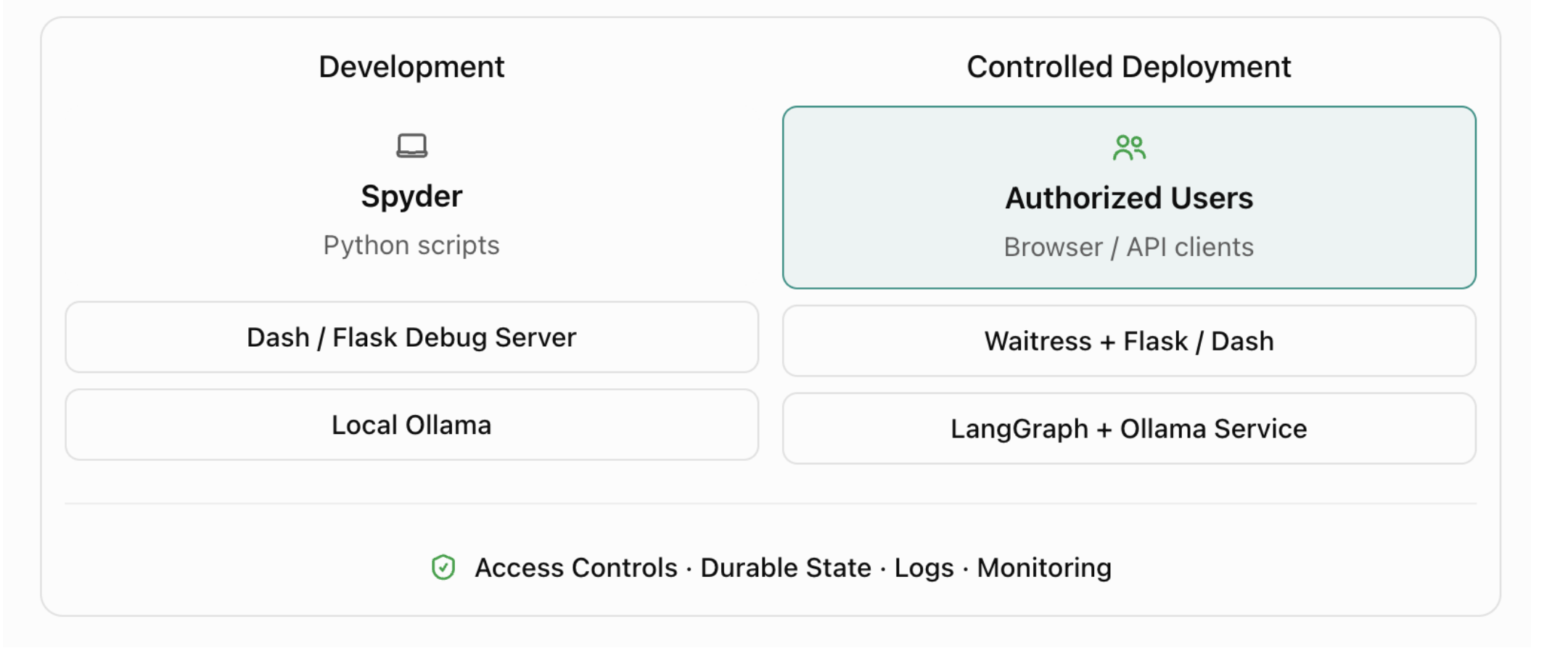


Comparison:

Deployment option	Main advantage	Main limitation
Local	Simple setup and direct debugging	Limited to workstation resources
Shared server	Centralized maintenance and GPU access	Requires network and capacity management
Containers	Reproducible application environments	Additional packaging and operational complexity

Reliability, Security, and Scalability

- Application Server: Use a production WSGI server such as Waitress rather than Flask's development server.
- Authentication and Authorization: Restrict API access and enforce permission checks before sensitive tool actions.
- Concurrency and Performance: Control simultaneous requests, model workload, and response timeouts.
- Persistent State: Store workflow states, checkpoints, and approvals so interrupted tasks can be recovered.
- Network Security: Restrict service exposure, protect internal databases and Ollama endpoints, and use HTTPS where appropriate.
- Logging and Error Handling: Record request identifiers, execution results, failures, and operational metrics without exposing sensitive information.


Development
Spyder
Python scripts
Dash / Flask Debug Server
Local Ollama
Controlled Deployment
Authorized Users
Browser / API clients
Waitress + Flask / Dash
LangGraph + Ollama Service
Access Controls · Durable State · Logs · Monitoring

Representative AI Platforms and Protocol

- **Amazon AWS Environment**
  - https://docs.aws.amazon.com/sagemaker/latest/dg/how-it-works-mlconcepts.html
- **C3.ai**
  - https://c3.ai/
- **Databricks**
  - https://www.databricks.com/
- **Dataiku**
  - https://www.dataiku.com/
- **DataRobot (together with Algorithmia)**
  - https://www.datarobot.com/
- **Domino Data Lab**
  - https://www.dominodatalab.com/
- **Telmai**
  - https://www.telm.ai/
- **Snowflake**
  - https://www.snowflake.com/en/

Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- **Dash HTML and Core Components**
- Dash Interactive Components
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

Dash: Basics

- Dash is an Open-Source **Python library** for creating reactive, Web-based applications.
- Those who use Python for data analysis, data exploration, visualization, modelling, instrument control, and reporting will find immediate use for Dash.
- Dash apps are composed of **two parts**.
  - The first part is the **layout** of the app and it describes what the application looks like.
  - The second part describes the **interactivity** of the application.

Dash: Basics

- Most html tags are provided as Python classes. For example, typing **html.H1(children='Hello Dash')** into your Dash script results in the HTML element **<h1>Hello Dash</h1>**.
- Dash offers two distinct component libraries. The code above comes from the **dash_html_components** library which has a component for every HTML tag, like the first-level heading **H1**.
- Another library, **dash_core_components**, offers higher-level, interactive components that are generated with JavaScript, HTML, and CSS through the React.js library.

Dash: Basics

- Python File Code Structure:



Dash: Layout

- **Example: HTML Layout**
- Let's create a simple HTML page that displays a bar chart. Run the following lines of code.

```
#%% SIMPLE DASH APP

from dash import Dash,dcc,html
import plotly.graph_objects as go

app=Dash(__name__)

fig=go.Figure([
    go.Bar(x=[1,2,3],y=[4,1,2],name="SF"),
    go.Bar(x=[1,2,3],y=[2,4,5],name="Montréal")
])

fig.update_layout(title="Dash Data Visualization",barmode="group")

app.layout=html.Div([
    html.H1("Hello Dash Rex Jalao"),
    html.P("Dash: A web application framework for Python."),
    dcc.Graph(id="example-graph",figure=fig)
])

if __name__ == "__main__":
    app.run(
        debug=True,
        jupyter_mode="external",
        use_reloader=False

    )
```

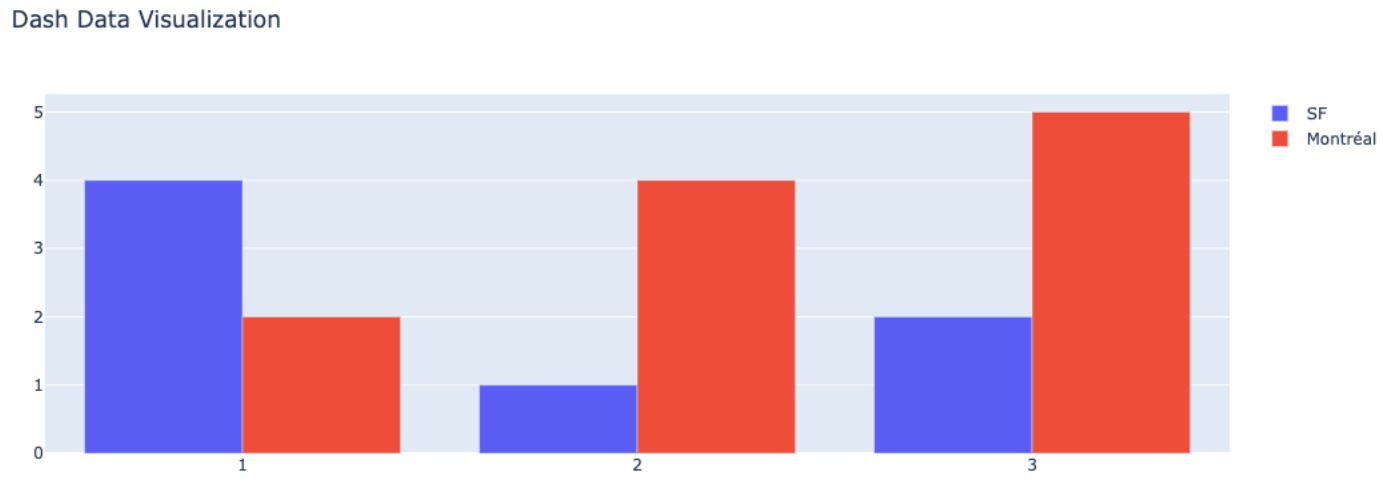
Dash: Layout

- Visit http://127.0.0.1:8050/ in your web browser.

```
5ver2.py'
Dash app running on http://127.0.0.1:8050/
```

Hello Dash Rex Jalao

Dash: A web application framework for Python.



Dash: Layout

- This active script that requires a local web server running in the background.
- If you should make changes to the layout run these lines of code and rerun the app.

```
#%%
from dash import jupyter_dash

server = jupyter_dash._servers.get(("127.0.0.1", 8050))

if server:
    server.shutdown()
    del jupyter_dash._servers[("127.0.0.1", 8050)]
    print("Dash server stopped.")
else:
    print("No Dash server is running on port 8050.")
```

Dash HTML Components

- Dash components are provided by two libraries: dash_html_components which we usually abbreviate to **html**, and dash_core_components, usually abbreviated to **dcc**.
- Normally, html components describe the layout of the page, including **placement and alignment** of different graphs.
- Instead of writing HTML or using an HTML templating engine, you **compose your layout** using Python structures with the dash-html-components library.
- Reference:

https://dash.plotly.com/dash-html-components

Dash HTML Components

- Common components include:
  - **html.Div**([section])applies CSS to section of page
    -
  - **html.P**() paragraph
    -
  - **html.H1**('text') heading (level 1)
    - <h1> </h1>
- HTML elements and Dash classes are mostly the same but there are a few key differences:
  - The **style** property is a dictionary
  - Properties in the style dictionary are **camelCased**
  - The *class* key is renamed as **className**
  - Style properties in pixel units can be supplied as just numbers without the *px* unit

UP NEC

Dash HTML Components

- Structure of a Div:

```
html.Div(
      children=[
            <More Elements>
      ],
      style={},
      id="name_of_div",
)
```

Dash HTML Components

```
html.Div([
    html.Div('Example Div', style={'color': 'blue', 'fontSize': 14}),
    html.P('Example P', className='my-class', id='my-p-element')
], style={'marginBottom': 50, 'marginTop': 25})
```

- Is equivalent to:

```
<div style="margin-bottom: 50px; margin-top: 25px;">

    <div style="color: blue; font-size: 14px">
        Example Div
    </div>

    <p class="my-class", id="my-p-element">
        Example P
    </p>

</div>
```

Dash HTML Components

- Dash HTML Tables

```
html.Table([
        html.Tr([
                html.Td("Cell 1"),
            html.Td("Cell 2"),
        ]),
        html.Tr([
                html.Td("Cell 3"),
            html.Td("Cell 4"),
        ]),
])
```

- Is equivalent to:

Cell 1	Cell 2
Cell 3	Cell 4

Dash Core Components

- Dash ships with supercharged components for interactive user interfaces.
- A core set of components, written and maintained by the Dash team, is available in the dash-core-components library.

Dash Core Components

- **Example: Core Components**
- Run the following lines of code to illustrate three dash core components.

```
from dash import Dash,dcc,html

app=Dash(__name__)

cities=[
    {"label":"New York City","value":"NYC"},
    {"label":"Montréal","value":"MTL"},
    {"label":"San Francisco","value":"SF"}
]

app.layout=html.Div([
    html.Label("Dropdown"),
    dcc.Dropdown(options=cities,value="MTL"),

    html.Label("Multi-Select Dropdown"),
    dcc.Dropdown(options=cities,value=["MTL","SF"],multi=True),

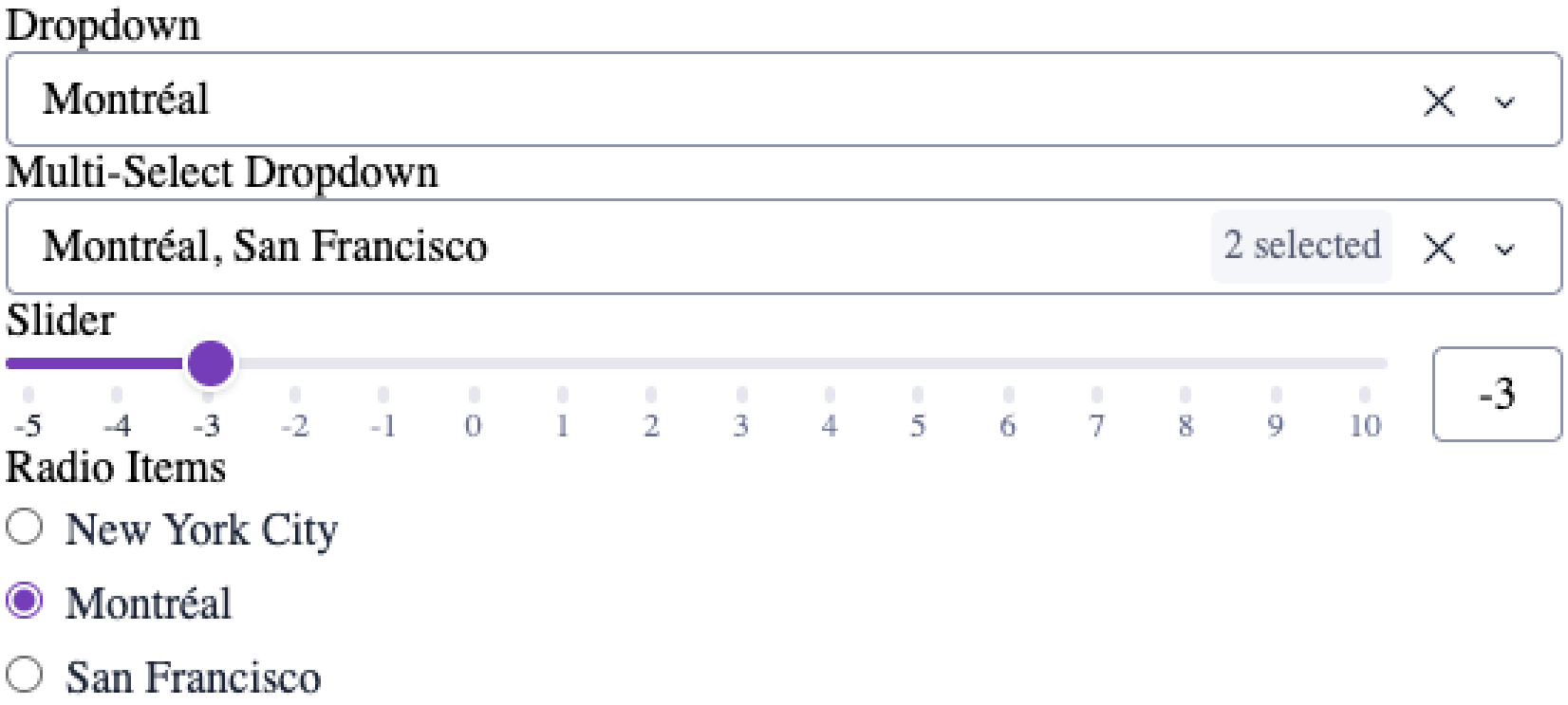
    html.Label("Slider"),
    dcc.Slider(
        min=-5,max=10,step=0.5,
        marks={i:str(i) for i in range(-5,11)},
        value=-3
    ),

    html.Label("Radio Items"),
    dcc.RadioItems(options=cities,value="MTL")
],style={"width":"50%"})

if __name__=="__main__":
    app.run(
        debug=True,
        jupyter_mode="external",
        use_reloader=False

    )
```

Core Components

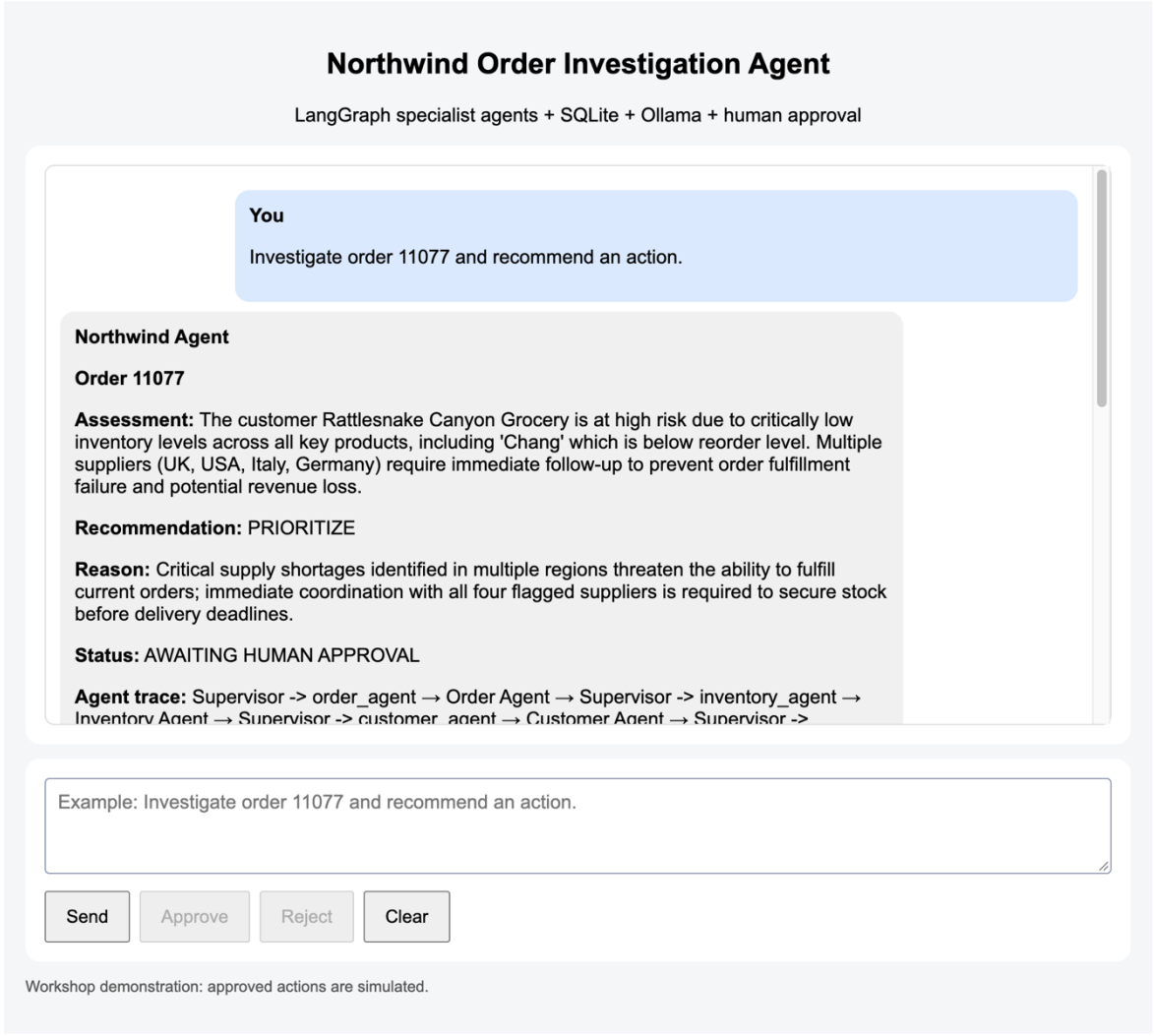


Core Components

- Example showed three built-in components:
  - Dropdown
  - Slider
  - Radio Items
- Many others are available, such as:
  - Buttons
  - RangeSlider
  - Input
  - Textarea
  - Checklist (horizontal and vertical checkboxes)
  - DatePicker
- https://dash.plotly.com/dash-core-components

HTML and Core Components

- **Exercise 1 —** Create a Dash Chatbot Layout



ERLJalao Copyright UPN

Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- Dash HTML and Core Components
- **Dash Interactive Components**
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

Interactive Components

- This session describes how to make your Dash apps using **callback functions**
- Callback functions make your Dash apps **interactive**.
- Python functions that are automatically called by Dash whenever an input component's property changes.

Interactive Components

- Interaction is done through a **callback function**
- Takes **inputs** and produces **outputs**
- Needs to link input components through an “id” and “property” **pair** to an output component through an “id” and “property” pair.
- There can only be one callback function that **outputs to a component** “id” and “property” pair.
- Function is executed whenever an input property is **changed**
- **State** of a component can be tracked without execution

Interactive Components

```
@app.callback(
      [ <List of Outputs> ],
      [ <List of Inputs> ],
      [ <List of States> ]
            )
def function_name(input_values, state_values):
      #do stuff here
      return [output_values]
```

- Note the following consistencies:

$$\#\ of\ \textbf{\textit{Inputs}} + \#\ of\ \textbf{\textit{States}} = \#\ of\ \texttt{input_values}\ +\ \texttt{state_values}$$

$$\#\ of\ \textbf{\textit{Outputs}} = \#\ of\ \texttt{return}\ \texttt{output_values}$$

Interactive Components

```
@app.callback(
      [Output(component_id='output_component',
      component_property='output_property')],
      [Input(component_id=' input_component',
      component_property='input_property')],

      [State(component_id='state_component',
      component_property='state_property')]
            )
def function_name(input_value, state_value):
      #do stuff here

      return [output_property]
```

Example Callback

- **Example: Callbacks**
- In the session python file, execute the following lines of code:

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/g

external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']

app = Dash(__name__, external_stylesheets=external_stylesheets)

app.layout = html.Div([
    dcc.Graph(id='graph-with-slider'),
    dcc.Slider(
        id='year-slider',
        min=df['year'].min(),
        max=df['year'].max(),
        value=df['year'].min(),
        marks={str(year): str(year) for year in df['year'].unique()},
        step=None
    )
])


@app.callback(
    [Output('graph-with-slider', 'figure')],
    [Input('year-slider', 'value')])
def update_figure(selected_year):
    filtered_df = df[df.year == selected_year]

    fig = px.scatter(filtered_df, x="gdpPercap", y="lifeExp",
                     size="pop", color="continent", hover_name="country",
                     log_x=True, size_max=55)

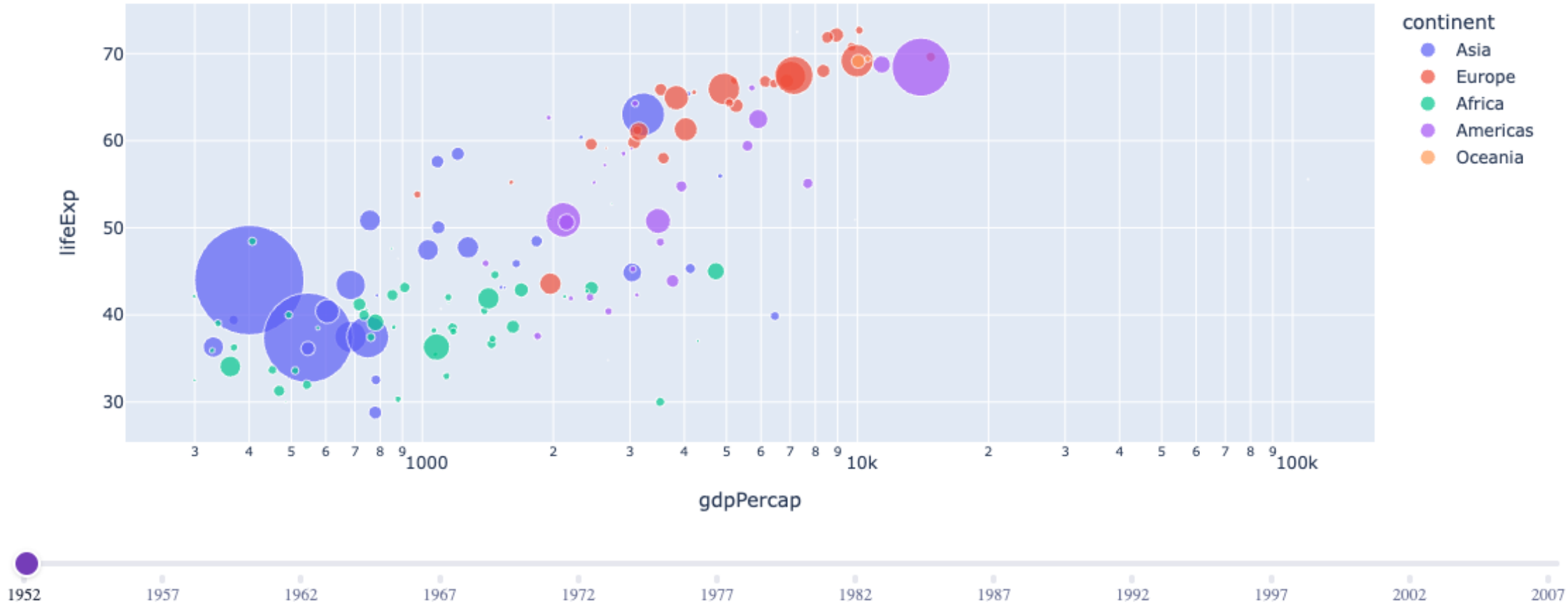
    fig.update_layout(transition_duration=500)

    return [fig]


if __name__ == '__main__':
    app.run(
        debug=True,
        jupyter_mode="external",
        use_reloader=False

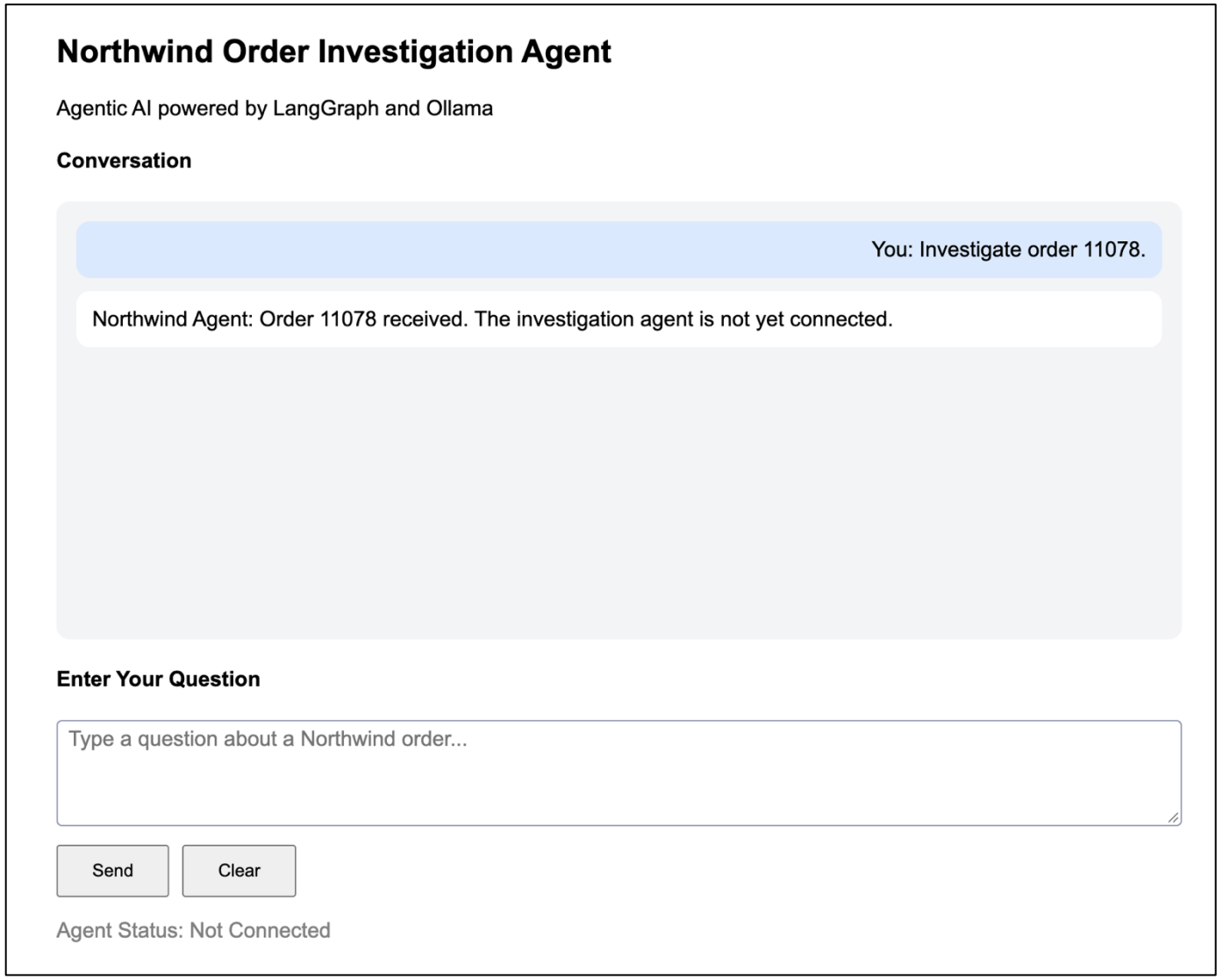
    )
```

Example Callback



Dash Callbacks

- **Exercise 2:** Callbacks



Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- Dash HTML and Core Components
- Dash Interactive Components
- **Dash Advanced Components**
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

Multiple Inputs

- For functions that require **multiple conditions** and **inputs** such that it will be executed we extend the Input component of the decorator function as a list:

```
[Input(component_id=‘ input_component1’,
      component_property=‘input_property1’),
 Input(component_id=‘ input_component2’,
      component_property=‘input_property2’),
],
```

- Inputs of the function should also **adjust accordingly**

```
def function_name(input_value1, input_value2):
```

Multiple Inputs

- **Example: Callbacks**
- In the session python file, execute the following codes:

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd

external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']

app = Dash(__name__, external_stylesheets=external_stylesheets)

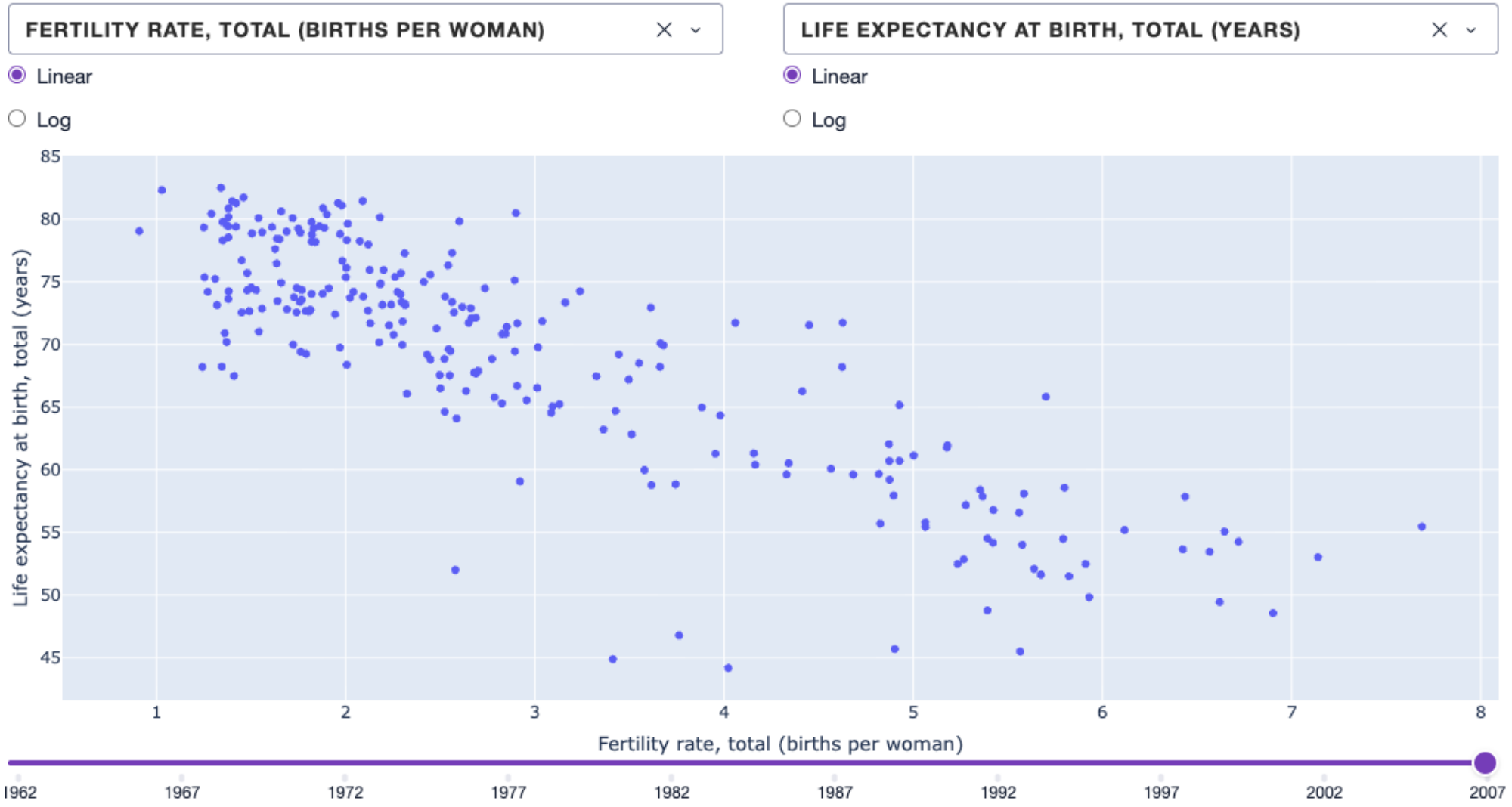
df = pd.read_csv('https://plotly.github.io/datasets/country_indicators.csv')

available_indicators = df['Indicator Name'].unique()

app.layout = html.Div([
    html.Div([
        html.Div([
            dcc.Dropdown(
                id='xaxis-column',
                options=[{'label': i, 'value': i} for i in available_indicators
                value='Fertility rate, total (births per woman)'
            ),
            dcc.RadioItems(
                id='xaxis-type',
                options=[{'label': i, 'value': i} for i in ['Linear', 'Log']],
                value='Linear',
                labelStyle={'display': 'inline-block'}
            )
        ],
        style={'width': '48%', 'display': 'inline-block'}),

        html.Div([
            dcc.Dropdown(
                id='yaxis-column',
                options=[{'label': i, 'value': i} for i in available_indicators
                value='Life expectancy at birth, total (years)'
            ),
            dcc.RadioItems(
                id='yaxis-type',
                options=[{'label': i, 'value': i} for i in ['Linear', 'Log']],
                value='Linear',
                labelStyle={'display': 'inline-block'}
            )
        ], style={'width': '48%', 'float': 'right', 'display': 'inline-block'})
```

Multiple Inputs



Chained Callbacks

- You can also **chain** outputs and inputs together: the output of one callback function could be the input of another callback function.
  - The second function will wait until the first function is done executing
- This pattern can be used to create dynamic UIs where one input component updates the available options of the next input component.



Chained Callbacks

- **Example: Callbacks**
- In the session python file, find and execute the following codes:

```
from dash import Dash, dcc, html, Input, Output

external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']

app = Dash(__name__, external_stylesheets=external_stylesheets)

all_options = {
    'America': ['New York City', 'San Francisco', 'Cincinnati'],
    'Canada': [u'Montréal', 'Toronto', 'Ottawa']
}
app.layout = html.Div([
    dcc.RadioItems(
        id='countries-radio',
        options=[{'label': k, 'value': k} for k in all_options.keys()],
        value='America'
    ),

    html.Hr(),

    dcc.RadioItems(id='cities-radio'),

    html.Hr(),

    html.Div(id='display-selected-values')
])
```

Chained

```
@app.callback(
    [Output('cities-radio', 'options')],
   [ Input('countries-radio', 'value')])
def set_cities_options(selected_country):
    list = [{'label': i, 'value': i} for i in all_options[selected_country]]
    return [list]

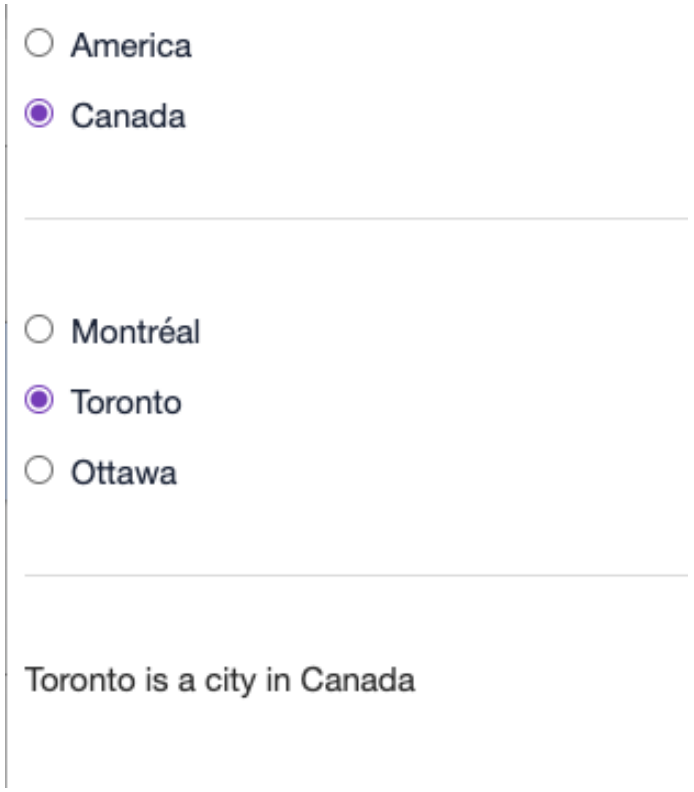

@app.callback(
    [Output('cities-radio', 'value')],
    [Input('cities-radio', 'options')])
def set_cities_value(available_options):
    return [available_options[0]['value']]


@app.callback(
    [Output('display-selected-values', 'children')],
    [Input('countries-radio', 'value'),
    Input('cities-radio', 'value')])
def set_display_children(selected_country, selected_city):
    return [u'{} is a city in {}'.format(
        selected_city, selected_country,
    )]


if __name__ == '__main__':
    app.run(
        debug=True,
        jupyter_mode="external",
        use_reloader=False

    )
```

Chained Callbacks



Dash States

- Previous interactive examples seen how inputs immediately affect outputs.
- As values are entered/changed, the function is executed to reflect any changes.
- To **wait for other inputs** (e.g. forms) utilize the State Command.

```
@app.callback(
      [Output(component_id='output_component',
      component_property='output_property')],
      [Input(component_id=' input_component',
      component_property='input_property')],

      [State(component_id='state_component',
      component_property='state_property')] )
def function_name(input_value, state_value):
      #do stuff here

      return [output_property]
```

Dash States

- **Example: Callbacks with States**
- In the session python file, find and execute the following codes:

```
from dash import Dash, dcc, html, Input, Output,State

app =Dash()

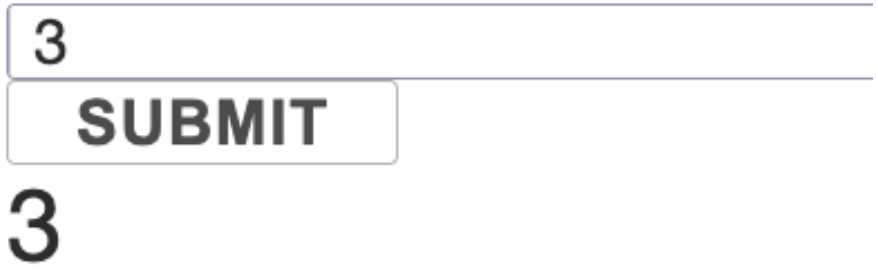
app.layout = html.Div([
    dcc.Input(
        id = 'number-in',
        value = 1,
        style={'fontSize':28}
        ),
    html.Button(
        id='submit-button',
        n_clicks=0,
        children = 'Submit',
        style={'fontSize':28}
        ),
    html.H1(id='number-out')
])

@app.callback(
    [Output('number-out','children')],
    [Input('submit-button','n_clicks')],
    [State('number-in','value')]
    )
def output(n_clicks, number):
    return [number]

if __name__ == "__main__":
    app.run_server()
```

Dash States

- **Example: Callbacks with States**



Callback Context

- In addition to event properties like **n_clicks** that change whenever an event happens there is a global variable **dash.callback_context**, available only inside a callback. It has properties:
  - triggered: list of changed properties.
  - inputs and states: allow you to access the callback params by id and prop instead of through the function args.
    - These have the form of dictionaries { 'component_id.prop_name': value }

Callback Context

- **Example: Callbacks with Context**
- In the session python file, find and execute the following codes:

```
from dash import Dash, dcc, html, Input, Output, State, ctx

external_stylesheets = ["https://codepen.io/chriddyp/pen/bWLwgP.css"]

app = Dash(__name__, external_stylesheets=external_stylesheets)

app.layout = html.Div([
    html.Div('Convert Temperature'),
    'Celsius',
    dcc.Input(
        id="celsius",
        value=0.0,
        type="number"
    ),
    ' = Fahrenheit',
    dcc.Input(
        id="fahrenheit",
        value=32.0,
        type="number",
    ),
])

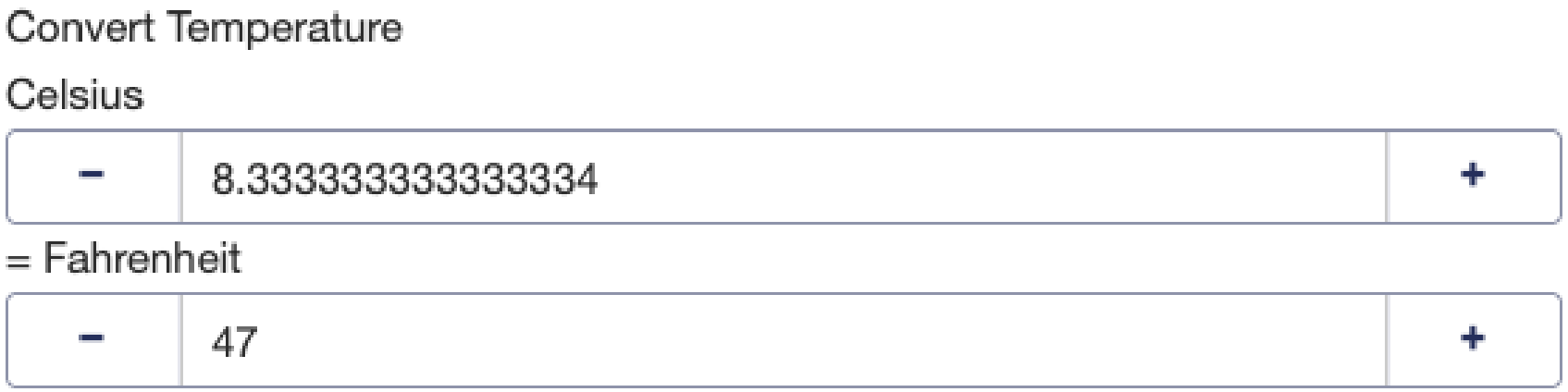
@app.callback(
    Output("celsius", "value"),
    Output("fahrenheit", "value"),
    Input("celsius", "value"),
    Input("fahrenheit", "value"),
)
def sync_input(celsius, fahrenheit):

    input_id = ctx.triggered_id
    if input_id == "celsius":
        fahrenheit= None if celsius is None else (float(celsius) * 9/5) + 32
    else:
        celsius = None if fahrenheit is None else (float(fahrenheit) - 32) * 5/9
    return celsius, fahrenheit

if __name__ == "__main__":
    app.run(
        debug=True,
        jupyter_mode="external",
        use_reloader=False

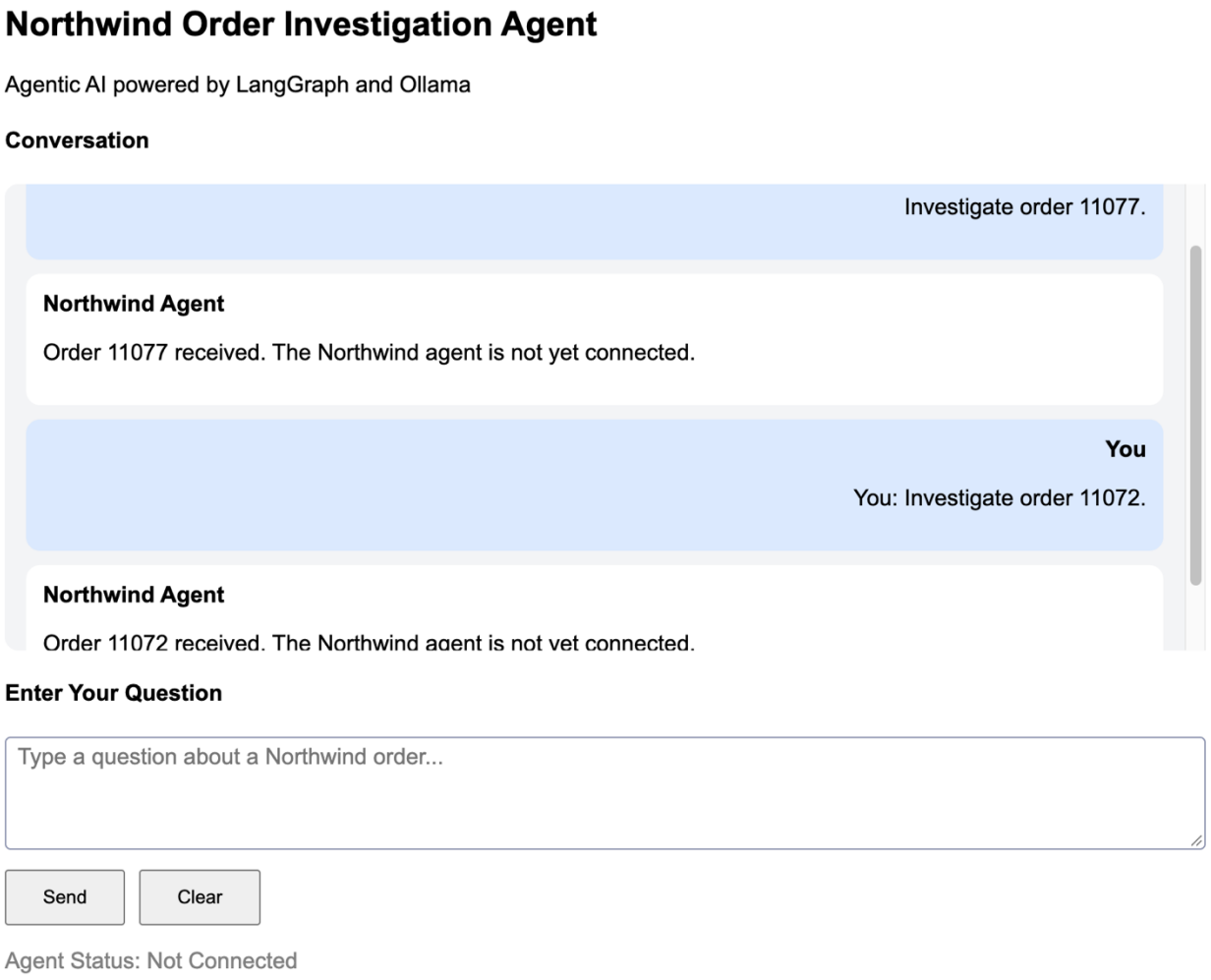
    )
```

Callback Context



Advanced Dash

- **Exercise 4:** Chatbot History

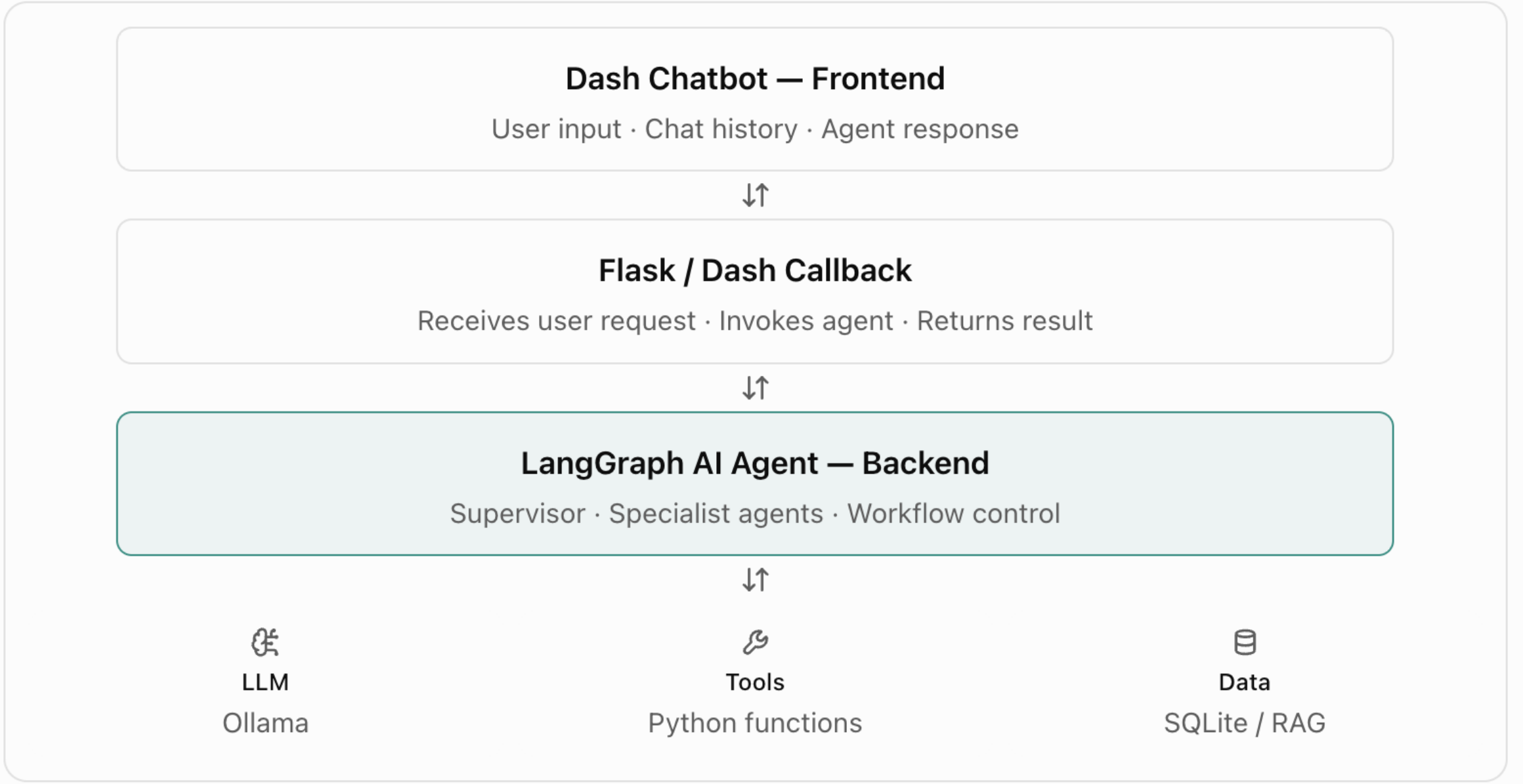


Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- Dash HTML and Core Components
- Dash Interactive Components
- Dash Advanced Components
- **Connecting Dash to an AI Agent**
- Case Study and Final Project Presentation

Connecting Dash to an AI Agent

- Dash provides the web interface through which users submit questions and view results.
- Flask provides the underlying web server that processes Dash callback requests.
- A Python callback passes the user's request to an existing AI agent.
- The agent executes its workflow using an LLM, tools, and external data sources.
- The completed result is returned to Dash for display.

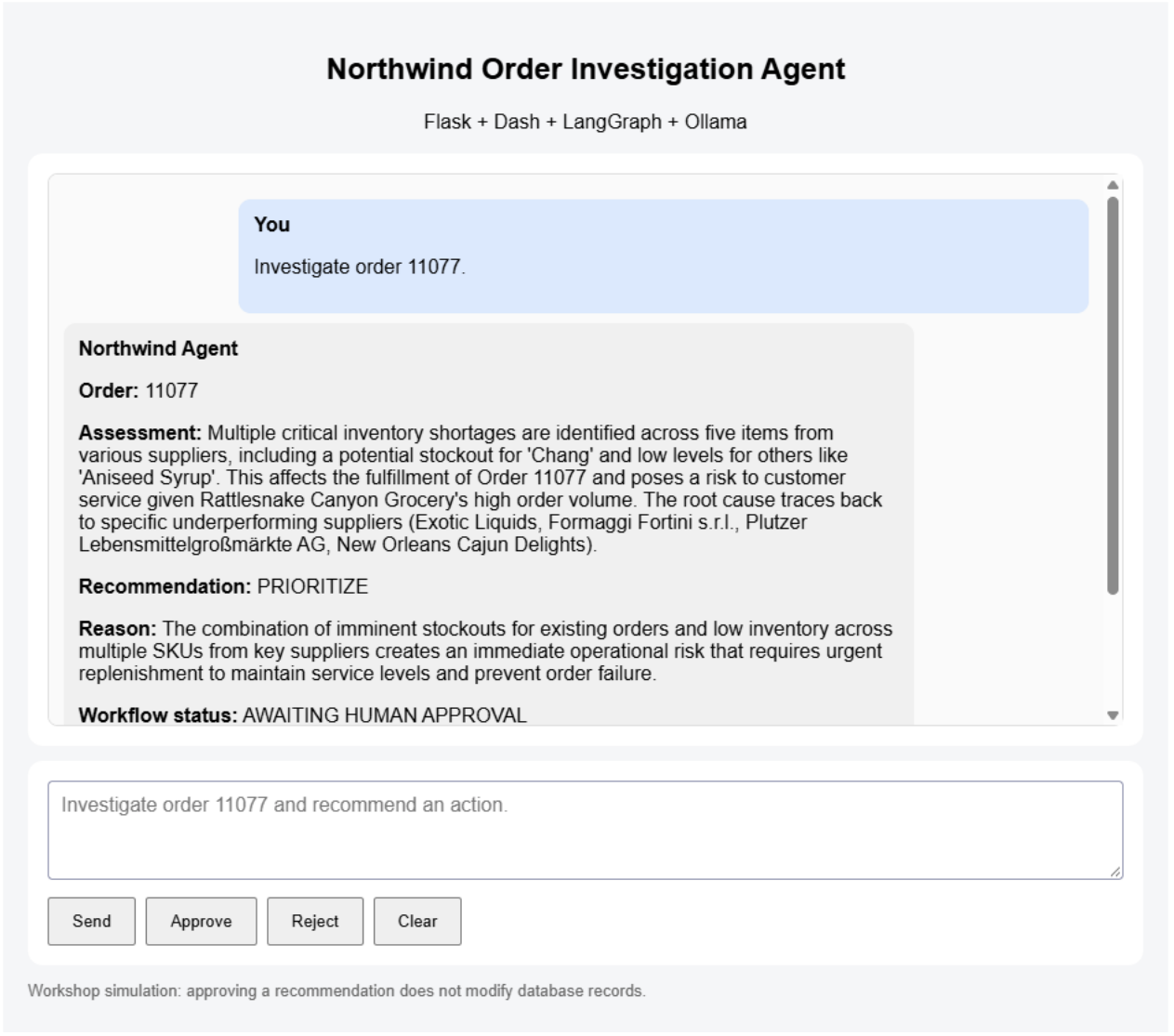

Dash Chatbot — Frontend
User input · Chat history · Agent response
Flask / Dash Callback
Receives user request · Invokes agent · Returns result
LangGraph AI Agent — Backend
Supervisor · Specialist agents · Workflow control
LLM
Ollama
Tools
Python functions
Data
SQLite / RAG

Workflow

- A callback receives a user question through Dash Input and State.
- The callback invokes an existing Python agent function or LangGraph workflow.
- The agent produces a response, execution trace, and workflow status.
- The callback extracts the relevant results and updates the chatbot interface.
- The agent can be replaced or upgraded without redesigning the Dash layout.

Dash With Agent

- **Exercise 5:** With Agent



Outline for this Session

- Introduction to LLMOps
- Architecture and Version Management
- Deployment
- Dash HTML and Core Components
- Dash Interactive Components
- Dash Advanced Components
- Connecting Dash to an AI Agent
- Case Study and Final Project Presentation

Final Project

- Present your agentic AI solution with an Agentic AI interface
  - Present 5 Slides
  - 1. Business Case
  - 2. Business Flowchart
  - 3. Final Architecture (See Slide 11)
  - 4. Evaluation/Evals Used
  - 5. Quick Demo
- Put all your names in the title slide and send to eljalao@up.edu.ph